Metric Justifications

Order Processing Duration

`nopcommerce.order.processing_duration` (histogram, milliseconds)

This metric would tell an operator that the checkout pipeline is degrading before users start seeing errors.

ASP.NET Core already tracks HTTP request duration, but that number includes middleware, serialization, view rendering, and other overhead that has nothing to do with the actual order logic. This metric measures only the time spent inside `PlaceOrderAsync`, which is where validation, payment, persistence, and inventory adjustment happen.

If this metric's p95 starts climbing while HTTP latency stays flat, the operator immediately knows the problem is inside the order pipeline, not the web layer. That narrows the investigation to payment gateways, database contention, or inventory logic. Without this separation, a latency spike could mean anything.

Tags: `payment_method`, `status` (success/failure).

Inventory Adjustment Failures

`nopcommerce.inventory.adjustment_failures` (counter)

This metric would tell an operator that customers are buying products that aren't actually available, before refund requests start piling up.

Under normal conditions this counter sits at zero. Any increment is immediately actionable: it means either the product catalog has stale availability data, or concurrent orders are racing on low-stock items. The `product_id` tag tells the engineer exactly which product to investigate and whether to disable it or restock.

This catches overselling at the moment it happens, not hours later when a customer opens a support ticket.

Tags: `product_id`, `reason` (out_of_stock / unknown).

Note on `product_id` cardinality: this tag is bounded by catalog size, not by request volume. A typical nopCommerce store has hundreds to low thousands of products, which keeps the label set manageable. For a store with tens of thousands of SKUs, this would need revisiting (e.g., aggregating by category instead). The operational benefit of knowing exactly which product is overselling justifies the cardinality cost at this scale.

Payment Gateway Latency

`nopcommerce.payment.gateway_duration` (histogram, milliseconds)

This metric would tell an operator whether a checkout slowdown is their problem or the payment provider's problem.

Payment is the only step in the order flow that calls an external system. It has fundamentally different failure modes: network timeouts, rate limiting, provider outages. If this histogram spikes alongside order processing duration, the root cause is the gateway. If only order processing duration spikes, the problem is internal. That distinction changes the entire incident response: you either escalate to the payment provider or you look at your own database and services.

Tags: `payment_method`, `status` (success/failure).

What we deliberately excluded from metric tags

We excluded `customer_id`, `email`, `order_total`, `ip_address`, and any other customer-identifying attribute from all metric tags. The reasons:

- **PII risk.** The slides define `user_id` as a metric label as "PII in disguise." Under GDPR, metrics with customer identifiers become personal data subject to retention, access control, and deletion requirements. Metric backends (Prometheus, for example) are not designed for GDPR deletion requests.
- **Cardinality explosion.** Tags like `customer_id` create one time series per customer. With thousands of concurrent users, this blows up storage and query performance in Prometheus. The OpenTelemetry documentation and course slides both warn against unbounded label values.
- **You can still get per-customer detail when you need it.** The trace (linked by trace_id) carries the full span hierarchy for a single request. If an operator sees a payment duration spike in the metric, they switch to Jaeger and filter by the relevant time window. No need to pollute aggregated metrics with per-customer dimensions.

The tags we do include (`payment_method`, `status`, `product_id`, `reason`) are all low-cardinality, operationally useful, and free of PII.

Privacy strategy: where redaction happens

Metric tags are chosen so that no redaction is needed on the metrics pipeline. PII never enters metric labels in the first place.

For traces, the situation is different. ASP.NET Core auto-instrumentation and nopCommerce's internal logging can attach customer-related attributes to spans (e.g., `customer_ip`, `email`). We handle this at the SDK layer using a custom `BaseProcessor<Activity>` called `PiiSanitisationProcessor`. It runs `OnEnd` before export and replaces any span attribute matching a PII pattern (email, name, phone, address, card, password, etc.) with `[REDACTED]`.

We chose SDK-layer redaction (instead of Collector or Storage-layer) because:

- **Data never leaves the process unsanitised.** Even if the collector is misconfigured or the storage backend is compromised, PII was already stripped.
- **Strongest guarantee.** The slides rank SDK/app redaction as the layer with the strongest privacy benefit, at the cost of requiring code changes.

- **Tradeoff accepted.** We lose the ability to recover redacted fields for debugging. If an incident requires the actual email address, the operator would need to query the application database directly, which has its own access controls. This is a deliberate choice: observability data should not become a second database of customer information.

Why inline instrumentation (and not decorator, middleware, or events)

We instrument by adding `StartActivity()` and `Stopwatch` calls directly inside the service methods. This mixes observability code into business logic, which is a cohesion tradeoff (the slides would classify this as adding a second "reason to change" to each service). We considered three alternatives:

- **Decorator pattern (OCP-aligned):** wrap `IOrderProcessingService` with an `InstrumentedOrderProcessingService` that adds spans around each call. Business code stays untouched. But nopCommerce uses `virtual` methods on concrete classes rather than interface-based DI for these services, which makes this impractical without refactoring the DI registration.
- **Middleware (sequential cohesion):** natural for HTTP-level metrics, and we already get this from `AddAspNetCoreInstrumentation()`. But middleware can only observe the full HTTP request; it cannot measure individual steps inside the order flow (validation, payment, persistence, inventory).
- **IConsumer event handlers (nopCommerce's own pattern):** react to `OrderPlacedEvent` to record metrics. This is loosely coupled, but you lose the timing granularity: you cannot measure duration if you only see the "done" event, and you cannot capture sub-steps at all.

We chose inline instrumentation because the alternative (decorator) would require refactoring nopCommerce's DI setup, which is not the point of this assignment. The tradeoff is that observability code is coupled to business logic, but the instrumentation is minimal (one line per method) and easily removable.